



시스템 아키텍처 및 Framework 기능

문서번호	InfinitPINN_시스템아키텍처_Framework_기능_v0.2.pptx
문서버전	Rev 0.2
작성일자	2026. 06. 05
작성자	김찬석



Document History

InfiniTINN Platform Development

■ 생성일자

버전	일자	구분	내역	담당자
-	2026-05-17	Framework	Initial	김찬석

■ 변경일자

버전	일자	구분	내역	담당자
v0.1	2026-05-28	Framework	Framework 아키텍처 및 설명	김찬석
v0.2	2026-06-05	Framework	Persistence, interface, log 추가	김찬석

InfiniTINN Platform Development



- 1 요구사항 분석
- 2 프레임워크 아키텍처
- 3 Backend framework 기능
- 4 Persistence
- 5 Interface
- 6 사용자 및 권한 관리
- 7 감사 추적

1. 요구사항 분석

필요 기능	필요 기능 상세	구현 방안 및 구현 내용
Zero Trust 기반 권한 관리	○ 제로 트러스트의 3대 핵심 원칙 - Zero Trust(Never Trust, Always Verify) - 명시적 검증 (Verify Explicitly) - 최소 권한 부여 (Least Privilege Access) - 침해 가정 (Assume Breach)	• 사용자(User)를 등록 • Role을 등록하고, Role과 User를 Mapping 하여 권한 Group을 한다 • Role은 다른 Role에 속할 수 있다, 즉 Role은 사용자를 Mapping 하거나, 사용자의 권한 그룹(Role)을 또 Mapping 할 수 있다 • Permission를 이용하여 기능(Menu, Action, API 등)을 정의하고 Role과 Permission를 Mapping하여 개인별 권한을 관리 • <i>실행 시 권한을 Check 할 수도 있다-실행 요청(Client) 기능에 대한 권한 Check</i>
기능 접근 권한 (Menu Access Role) 관리	○ 사용자의 직무(RBAC: Role-Based Access Control)에 따른 플랫폼 내 기능에 대한 접근을 통제 - 관제 및 모니터링 (MICC) - 자산 및 기준 정보 관리 (DTAA) - 시스템 설정 및 운영 - 권한 그룹 생성 - 제어를 위한 Action단위 권한 설정	
AI Agent 사용 권한	○ AI Agent 사용 권한 (Agent Execution & Interaction Role) 지능형 에이전트(PI-LAM, LLM 등)를 호출하고 그 결과값을 현장에 반영할 수 있는 통제권을 의미 - 에이전트 호출 권한 - 제어 승인 권한 (Sim-to-Real) - 프롬프트 및 레시피 수정	• <i>T.B.D</i>
데이터 활용 권한	○ 데이터 활용 권한 (Data Governance & Access Role) - 컨텍스트별 데이터 격리 - 데이터 다운로드 및 반출 - 지식 자산(MIM) 기여 권한	• <i>T.B.D</i>

1. 요구사항 분석

필요 기능	필요 기능 상세	구현 방안 및 구현 내용
데이터 보호 및 암호화 - DB	○ 데이터베이스 저장 구간: 선택적 암호화 및 OS 레벨 암호화 상용 DB(Oracle 등)의 고가 TDE 라이선스를 구매하지 않고 오픈소스 DB(PostgreSQL/TimescaleDB)에서 암호화 - 무료 OS 레벨 암호화 (LUKS / dm-crypt) - 데이터 중요도에 따른 '선택적 암호화 (Selective Encryption)'	• T.B.D
데이터 보호 및 암호화 - 데이터 전송	○ 데이터 전송 구간: 최신 프로토콜 및 하드웨어 가속 활용 네트워크 통신 시 오픈소스 인증서와 최신 표준을 사용 - 인증서 운영 (Internal CA / Let's Encrypt) - TLS 1.3 적용 - Edge 환경 최적화 알고리즘 (ChaCha20-Poly1305)	• T.B.D
데이터 보호 및 암호화 - 데이터 마스킹 및 익명화	○ 동적 데이터 마스킹 (Dynamic Data Masking): - 데이터베이스에는 원본을 그대로 저장하되, 사용자 화면이나 API로 데이터를 내보낼 때만 실시간으로 * 처리를 하는 방식 - 장점(비용 및 속도): 수학적 암호화/복호화 연산이 아니라 단순한 '문자열 치환(String Replace)'이므로 처리 지연(Latency)이 0에 가까움. Java Spring Boot의 인터셉터(Interceptor)나 API Gateway(Orchestrator)에서 몇 줄의 코드만으로 전사 적용이 가능 - 보안 효과: 권한이 없는 내부 직원의 화면 캡처나 어깨너머로 정보를 훑쳐보는 행위(Shoulder Surfing)를 완벽하게 차단 ○ 비식별화 처리 (De-identification): - 원천 데이터에서 개인을 특정할 수 있는 정보나 핵심 기밀을 다른 값으로 덮어쓰거나 뭉뚱그리는 방식 - 활용처 (AI Agent 학습): 이 기능은 특히 PI-LAM이나 LLM 에이전트에 매우 유용 AI가 과거 공정 데이터를 학습할 때, 굳이 '어느 고객사'의 '어느 제품'인지 정확한 명칭을 알 필요는 없음 물리적 수치(온도, 압력 등)만 중요 - 적용 예시: "A고객사 배터리 레시피" -> "Project X_Recipe" 로 식별자 자체를 변환하여 AI DB에 저장 - 보안 효과: AI 서버(또는 외부 LLM)가 해킹당하더라도, 유출된 데이터가 어떤 데이터인지 식별할 수 없어 기밀 유출을 방어	• T.B.D

1. 요구사항 분석

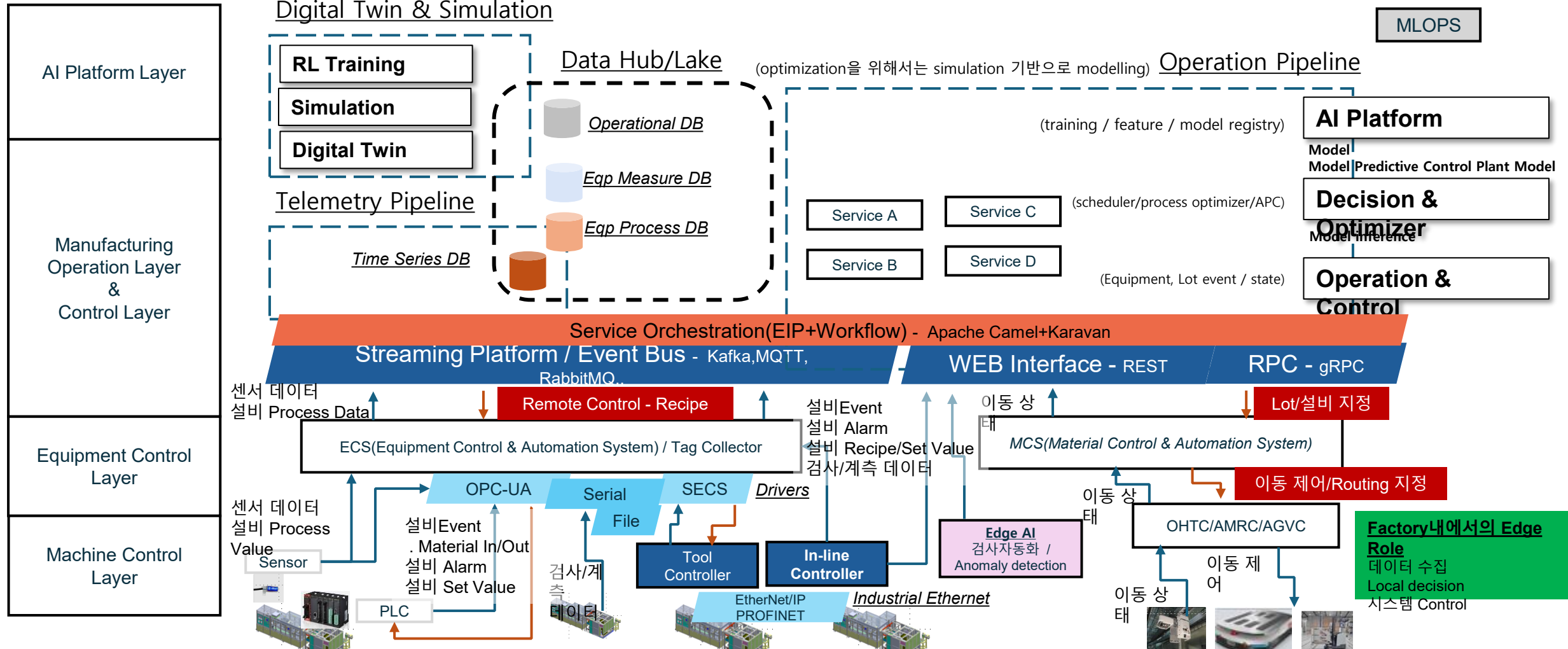
필요 기능	필요 기능 상세	구현 방안 및 구현 내용
가시성과 감사 추적	- 가시성(Observability)과 감사 추적(Audit Trail) - 핵심 로깅 대상: AI 에이전트 실행 및 제어 명령 - 효율적 조회를 위한 MSA 분산 추적 (Distributed Tracing) - 고성능 로그 수집 및 저장 아키텍처 - MICC (관제 센터) 연동 사후 분석 화면	
가시성과 감사 추적 - AI 에이전트 실행 및 제어 명령	AI의 의사결정 과정과 현장 제어 명령은 법적 증빙(Compliance) 수준으로 상세히 기록되어야 함 - AI 추론 컨텍스트 (Explainable AI Log): - AI(PI-LAM)가 판단을 내릴 당시 참조한 원천 데이터(센서 값, 레시피 등)의 스냅샷. - 물리 분석(PVSA) 결과값 및 AI가 특정 제어 명령을 선택한 논리적 근거(확률값, 기대 효과 등). - 제어 명령 발행 및 실행 이력 (Control Trail): - 에이전트가 어떤 설비(Target)에 어떤 파라미터(Set-point)를 변경하라고 명령했는지 기록. - MIAM(권한 관리) 연동: AI의 제안을 최종적으로 '승인' 또는 '반려'한 담당자의 정보와 승인 시점. - 현장 반영 확인 (Acknowledge Log): - 명령이 실제 PLC/Edge로 전송되었는지, 설비가 정상적으로 수신하여 상태를 변경했는지에 대한 왕복(Round-trip) 결과.	- 감사 추적을 위해 'trace_id'를 활용하여 하나의 요청 cycle에 id를 부여하여 trace를 할 수 있게 한다 'trace_id'는 Client->Server로 요청 시 메시지 header에 담아 보내거나, Server에서 생성한다 - 감사 추적을 위해서는 일반적인 시스템 log와 감사 추적용 log를 분리하여 관리한다 - MSA 아키텍처로 서비스가 업무별로 분리되어 있는 경우에는 일반적인 Log도 Process별로 분리됨 - Log는 1차로 log file로 관리 -> Logback - 감사 추적용 log는 user_behaviour_hist를 log file 또는 전용 Table로 관리한다
가시성과 감사 추적 -MSA 분산 추적 (Distributed Tracing)	마이크로서비스(MSA) 구조에서는 하나의 사건이 여러 서비스를 거쳐 발생, 파편화되지 않게 해야 함 - 고유 추적 ID (Correlation ID / Trace ID) 부여: - 현장에서 데이터가 수집되거나 AI가 분석을 시작하는 최초 시점에 고유 식별자(Trace ID)를 발급 - 이 ID가 Kafka 메시지 헤더나 gRPC 통신을 타고 모든 서비스를 관통하도록 설계 - 이벤트 타임라인 분석: - 관리자가 특정 불량이나 이상 상황 발생 시점의 Trace ID를 조회하면, 흩어져 있는 모든 마이크로서비스의 로그가 시간 순서대로(Timeline) 한 화면에 조립되어 "어디서 지연이 발생했고, 누가 문제를 유발했는지" 즉시 파악 가능하게 해야 함	- Table로 관리: Client의 요청 사항 및 수행 결과 저장, 업무용 DB Transaction과 별도로 Asynch하게 저장 - Log로 관리: Kafka/Logstash로 전달 후 별도로 관리 -> ELK 기반으로 관리 - 저장 내용: 각 업무별로 저장에 필요한 항목은 별도 정리 필요
가시성과 감사 추적 - 고성능 로그 수집 및 저장 아키텍처	서비스 처리 속도(Latency)에 영향을 주지 않으면서 방대한 로그를 수집해야 한다 - 비동기 로깅 및 Kafka 버퍼링: - 애플리케이션(Java/Python)은 로그를 파일이나 DB에 직접 쓰지 않고, 처리 속도가 매우 빠른 Kafka(메시지 큐)에 비동기로 던지기만 하고 본래 작업으로 돌아갑니다. (Zero-latency 전략) - ELK 스택 (Elasticsearch, Logstash, Kibana) 기반 검색 엔진: - Kafka에 쌓인 로그를 Logstash/Fluentd가 수집하여 Elasticsearch(또는 OpenSearch)에 저장 - Elasticsearch는 역인덱싱(Inverted Index)을 통해 초당 수십만 건의 로그 중 특정 에러 코드나 설비 이력을 0.1초 만에 검색	

2. Operation & Control 시스템 Architecture

InfiniTINN Platform
Development

시스템 구조 – Manufacturing Operation & Control Architecture

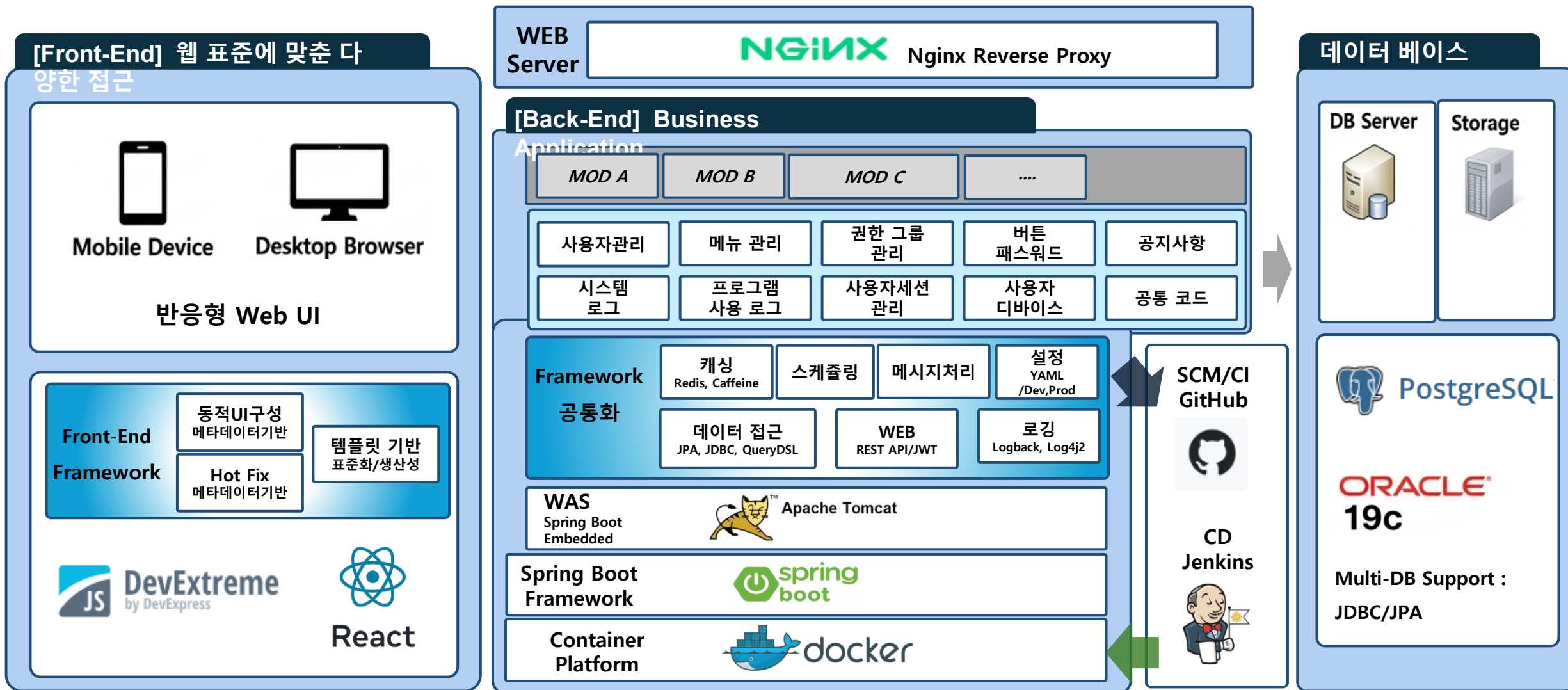
Smart Factory의 한계는 “데이터 중심 IoT 아키텍처”였고, 자율제조는 “Control 중심 Factory Intelligence 아키텍처”입니다.



POM: Production Operation Management, RMS : Recipe Management System, R2R : Run to Run Control, FDC : Fault Detection & Classification, EDC : Engineering Data Collection, SPC : Statistical Process Control, CMMS : Computerized Maintenance Management System, FEMS: Factory Energy Management System, OHTC : Overhead Hoist Transport Controller, AMRC: Autonomous Mobile Robot Controller, AGVC: Automated Guided Vehicle Controller

2. Framework 아키텍처(1)

InfiniTINN Platform
Development



SCM: Source Code Management, CI/CD : Continuous Integration, Continuous Delivery & Deployment, WAS: WEB Application Server

2. Framework 아키텍처(2)

InfinitPINN Platform
Development

요구사항		내용	
구분	내용	내용	비고
Framework	Spring Boot 4.x	Spring Boot 4.x	
Language	Java 21 LTS	Java 21 LTS	
WAS	Embedded Tomcat 10.x(Spring Boot)	Embedded Tomcat 10.x(Spring Boot)	Spring Boot Embedded
WEB	Nginx Reverse Proxy	Nginx Reverse Proxy	
DB	PostgreSQL + TimeScale DB		PostgreSQL + TimeScale DB
Front-End	React(반응형 UI)	React + 편의 기능	Front-End Template (JavaScript/DevExtream)
소스 관리	Git(GitLab Self-hosted)	Git(GitLab Self-hosted)	
배포	CI/CD(Jenkins/GitLab CI) + Docker	CI/CD(Jenkins/GitLab CI) + Docker	
보안인증	JWT + Spring Security 6.x		
아키텍처	Layered Architecture	Presentation - Business - Data Access	Presentation(Client) <-> Business 간 data 전달은 API 방식의 json 활용
	Layered Service Architecture	Worker > Rule > Operation Service	Worker Service : Client로 부터의 메시지 수신 Rule Service : Business Logic / 여러 Object 처리 Operation Service : Table 단위 또는 Child Table 포함한 단위의 업무 Business Logic * 조회성은 단순하게 구성 할 수 있음: Worker -> Query Rule Service * 기준정보(업무 로직이 없는 경우)도 단순하게 구성 : Client에서 Data set 구성(Json) -> Worker -> 기준정보 공통 Rule

3. Backend Framework 기능 - 전체

InfiniTINN Platform
Development



Backend Framework 기능 및 설명

순번	패키지	핵심 기능 (Core Function)	상세 설명 (Description)
1	aspect	Log, Exception 등	서비스 레이어의 로깅(AmsLogAspect), 성능 측정, 공통 예외 가로채기 등을 수행하는 감시자 역할
2	cache	데이터 캐싱 엔진	Caffeine 기반 L1 캐시 및 Redis 연동 관리. 시스템 마스터 데이터와 NLS 메시지의 초고속 조회를 담당
3	common.type	공통 상수 및 타입	프레임워크 전역에서 사용하는 Enum, 상수(Constants), 공통 인터페이스 규격을 정의
4	config	시스템 환경 설정	스프링 빈 설정, DB 커넥션 풀, 캐시 매니저, AOP 설정 등 프레임워크의 뼈대를 구성
5	context	실행 컨텍스트 관리	현재 스레드의 사용자 정보, 트랜잭션 ID(TID) 등 실행 시점의 상태 값을 보관하고 전파
6	exception	표준 예외 엔진	AmsException을 정의하여 에러 코드 기반의 체계적인 예외 처리 구조를 제공
7	handler	공통 핸들러	외부 인입 요청(HTTP, gRPC, Kafka)의 컨텍스트 주입부터 예외 수습 및 메시지 라우팅까지, 트랜잭션의 시작과 끝을 중앙 제어하는 실행 제어 사령탑
8	integration	외부 인터페이스 연동	메시징(Kafka/MQTT), 타 시스템 연동을 위한 규격 및 전송/수신 로직 관리
9	log	표준 로깅 컴포넌트	AmsLog를 통해 TID, User, Caller 정보가 포함된 고성능 표준 로그 포맷을 생성
10	nls	다국어 메시지 관리	MessageProvider를 통해 다국어 지원 및 {0} 형태의 동적 메시지 치환 기능을 제공
11	persistence	데이터 영속성 레이어	공통 Repository, Entity, 기본 CRUD JPA Wrapper 등 DB 저장 및 이력 자동화 엔진이 포함된 핵심 레이어
12	platform	query, 기준정보	일반적인 공통 기능 처리(기준정보 등)
13	support	프레임워크 보조 도구	EntityHelper, Reflection 도구 등 프레임워크 내부 구동을 돕는 유틸리티 모음
14	util	일반 유틸리티	날짜 처리, 문자열 파싱, 파일 입출력 등 범용적으로 사용되는 편의 도구

3. Backend Framework 기능 - handler

InfiniTINN Platform
Development



Backend Framework 중 handler package 세부 내용 기능 및 설명

순번	클래스명 (Class Name)	핵심 기능 (Key Function)
1	AmsTidFilter	HTTP 요청 최전방에서 트레이스 ID(traceId)를 발급 및 MDC 바인딩
2	AmsContextFilter	웹 요청에서 세션 정보(senderId, factoryId, lang)를 추출해 AmsUserContext에 주입
3	AmsGrpcInterceptor	gRPC 원격 호출 시 메타데이터에서 traceId 및 세션 정보를 추출해 스레드에 동기화
4	AmsKafkaInterceptor	카프카 메시지 컨슈머 시 레코드 헤더의 traceId를 고집어내어 비동기 스레드 컨텍스트 연계
5	AmsErrorProcessor	발생한 예외를 분석하여 표준 에러 코드로 변환하고 NLS 다국어 메시지로 치환 처리
6	GlobalExceptionHandler	Spring MVC(HTTP/REST) 전역에서 발생하는 예외를 가로채 표준 JSON 응답 규격으로 반환
7	GrpcGlobalExceptionHandler	gRPC 통신 중 발생한 에러를 캐치하여 gRPC 표준 상태 코드 및 메타데이터로 래핑 반환
8	KafkaGlobalExceptionHandler	카프카 리스너 내부 예외 발생 시 로깅 및 DLQ 연계 등 메시지 파이프라인 단절 방지
9	MessageResultHandler	비즈니스 로직의 최종 반환 데이터를 표준 공통 응답 포맷(Result, Message, Data)으로 패키징
10	WorkerRouter	인입된 제조 메시지의 커맨드나 타입을 분석하여 담당 MessageWorker로 동적 라우팅
11	MessageWorker	WorkerRouter에 의해 트리거되어 실제 비즈니스 핸들링을 수행하는 실행체 추상화 레이어
12	EntityTypeManager	핸들러 레이어에서 가변 메시지 직렬화 및 엔티티 메타 타입을 런타임에 식별·관리

3. Backend Framework 기능 – 공통 기능

InfiniTINN Platform
Development



Backend 공통기능 세부 내용 기능 및 설명(package:platform)

구분	기능	기능 설명
Auth	Login/Out	LogIn/Out, password 관리 rule, logIn hist 관리
query	조회용 query 실행	Query를 table에 저장 후 필요 시 query를 읽어 실행, result=DataTable(column정보 및 data)
	Update용 query 실행(jpa)	단순 1개의 CUD(update/update/insert)
	Update용 batch query실행(jpa)	복수 개의 row CUD – JPA 이용
	Update용 batch query실행(jdbc)	복수 개의 row CUD – JDBC 이용
	조회용 procedure실행	조회용 procedure 실행 – 조회되는 column 명시해야함
	Update용 procedure실행	Update용 procedure 실행
공통 기능	사용자	계정 생성, 상태 제어 등 마스터 사용자(User) 정보의 핵심 CRUD 처리
	사용자 그룹	사용자 그룹(Role)의 생성, 변경 및 그룹별 권한 매핑 제어
	Privilege	시스템 메뉴 및 기능별 접근 권한(Privilege)의 동적 할당 및 관리
	사용자-사용자그룹 연계	사용자-사용자그룹(Role 기준) mapping, 사용자그룹-사용자그룹 mapping : 한명의 사용자가 여러 그룹에 mapping 될 수 있다
	사용자그룹-Privilege연계	사용자그룹과 privilege 연계

4. Persistence : JPA, queryDSL vs JDBC 특징 및 용도

InfiniTINN Platform
Development



Persistence – DB 처리 방법별 비교

비교 항목	순수 JPA ORM	QueryDSL (JPA 기반)	JDBC / JdbcTemplate
핵심 가치	객체-DB 패러다임 일치, CRUD 생산성 극대화, 단건 조회/저장 표준화	동적 쿼리 명세화, Type-Safe(컴파일 시점 에러 체크), 복잡한 연동 쿼리 유지보수성	압도적인 성능, 하드웨어 레벨 제어, SQL 중심 제어
성능 오버헤드	영속성 컨텍스트(1차 캐시, 쓰기 지연) 관리 및 엔티티 생명주기 오버헤드 존재	JPA 오버헤드 + Q-Class 객체 생성 오버헤드 미비하게 추가 존재	거의 없음 (가장 빠름)
대량 등록/수정	✗ 비추천 (Identity 전략 시 Batch Insert 불가능, 건별 쿼리 유발 및 메모리 과부하)	✗ 비추천 (단건 루프 유발 및 벌크 연산 시 영속성 컨텍스트 미반영 리스크)	◯ 강력 추천 (batchUpdate를 통한 대량 벌크 처리 지원)
이력 관리 연동	◯ 자동 연동 (AmsBaseEntity 메커니즘 및 JPA 라이프사이클 리스너 완벽 활용)	◯ 자동 연동 (JPA 기반 엔티티 변경 시 AmsBaseEntity 메커니즘 활용 가능)	✗ 수동 구현 (직접 SQL로 감사/이력 쿼리를 따로 짜야 함)
사용처	기본 마스터 정보(MDM) CRUD, 단건 비즈니스 로직 처리, 도메인 주도 설계(DDD) 핵심 레이어	*Service, *Repository 레이어의 동적 조건 검색 및 화면 API 전체	설비 인터페이스(IF) 수집기, 대용량 로그 적재, 통계 배치 및 마이그레이션 모듈

4. Persistence : Entity - Repository



ORM을 사용하는 경우, 기본적으로 각 Table마다 Entity, Repository가 존재 해야 함

Table Layout

com_user
Columns (27)
user_id character varying(40)
user_name character varying(100)
password character varying(255)
email character varying(100)
pwd_changed timestamp without time zone
pwd_life integer
pwd_expiry_date timestamp without time zone
default_language character varying(40)
login_fail_count integer
is_locked character varying(40)
is_logged_in character(1)
last_login_time timestamp without time zone
last_api_time timestamp without time zone
state character varying(40)
event character varying(40)
event_user character varying(40)
event_time timestamp without time zone
event_comment character varying(200)
event_db_time timestamp without time zone
creator character varying(40)
create_time timestamp without time zone
last_modifier character varying(40)
last_modified_time timestamp without time zone
row_version bigint
tid character varying(40)
dtype character varying(40)
is_usable character varying(40)

Repository

@Repository 4 usages Kim, Chanseok

```
public interface UserRepository extends AmsRepository<User, String, UserHist> {
```

Entity

```
@Entity
@Table(name = "com_user")
@Getter @Setter @NoArgsConstructor @AllArgsConstructor
@SuperBuilder(toBuilder = true) // 부모 필드까지 빌더 지원
@EqualsAndHashCode(callSuper = false)
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "dtype", discriminatorType = DiscriminatorType.STRING)
@DiscriminatorValue("STD")
public class User extends AmsBaseEntity {
    @Id
    @Column(name = "user_id", length = 40)
    private String userId;

    @Column(name = "user_name", length = 100)
    private String userName;

    @Column(name = "password", length = 255)
    private String password;

    @Column(name = "email", length = 100)
    private String email;

    @Column(name = "pwd_changed")
    private LocalDateTime pwdChanged;

    @Column(name = "login_fail_count")
    private Integer loginFailCount;

    @Column(name = "is_locked", length = 40)
    private String isLocked;

    @Column(name = "is_logged_in", length = 1)
    private String isLoggedIn;

    @Column(name = "last_login_time")
    private LocalDateTime lastLoginTime;

    @Column(name = "last_api_time")
    private LocalDateTime lastApiTime;

    @Column(name = "state", length = 40)
    private String state;

    @Override
    public Object getEntityId() {
        return userId;
    }

    @Override
    public void setEntityId(Object key) {
        if (key instanceof String id) {
            this.userId = id;
        } else {
            throw new IllegalArgumentException("AMS Framework Error: Invalid Key T");
        }
    }

    @Override
    protected boolean isSnapshotEnabled() {
        return true;
    }
}
```

- Hist Table은 Entity는 필수, Repository는 Option
- 업무 로직이 없는 CRUD만 필요한 경우에는 Entity는 필수, Repository는 Option
- Custom Entity를 감안하여 필요한 annotation
 - @Inheritance(strategy = InheritanceType.SINGLE_TABLE)
 - @DiscriminatorColumn(name = "dtype", discriminatorType = DiscriminatorType.STRING)
 - @DiscriminatorValue("STD")
- 가능하면 Entity는 generator를 활용하여 생성
- 복합 Key인 경우 ID(Primary Key) 반드시 설정
- Hist를 저장하는 경우 : isSnapshotEnabled = true

4. Persistence : 공통 Repository(1)

InfiniTINN Platform
Development



표준 JPA Repository 기능 설명 - SimpleJpaRepository

분류	메서드명	반환 타입	설명	비고 / 쿼리 특징
저장 및 수정	save(S entity)	S	엔티티 1건을 저장하거나 수정	ID가 없으면 INSERT, 있으면 UPDATE 수행
	saveAll(Iterable<S> entities)	List<S>	여러 엔티티를 한 번에 저장/수정	내부적으로 루프를 돌며 save()를 건건이 호출
	saveAndFlush(S entity)	S	엔티티를 저장하고 즉시 DB에 반영	트랜잭션 종료 전 쿼리를 강제로 실행해야 할 때 사용
단건 조회	findById(ID id)	Optional<T>	ID로 특정 엔티티 1건을 조회	DB를 실제로 조회하며, 없을 경우 빈 Optional 반환
	getReferenceById(ID id)	T	ID로 엔티티의 프록시(가짜 객체)를 조회	실제 DB 조회를 지연(Lazy Loading)시킴 (↗ getById)
	existsById(ID id)	boolean	해당 ID를 가진 데이터가 존재하는지 확인	select count(*) 또는 select 1 쿼리 발생
목록 조회	findAll()	List<T>	테이블의 모든 데이터를 조회	데이터가 많을 경우 메모리 부하 주의
	findAllById(Iterable<ID> ids)	List<T>	여러 ID에 해당하는 엔티티 목록을 조회	WHERE id IN (?, ?, ...) 쿼리 발생
	findAll(Sort sort)	List<T>	정렬 조건(Sort)을 적용하여 전체 조회	ORDER BY 쿼리 추가
	findAll(Pageable pageable)	Page<T>	페이징 및 정렬 조건을 적용하여 조회	페이징 쿼리와 함께 전체 개수(Count) 쿼리 자동 실행
Delete	deleteById(ID id)	void	ID로 데이터 1건을 삭제	[주의] 내부적으로 findById 선조회 후 DELETE 수행
	delete(T entity)	void	특정 엔티티 객체를 삭제	영속성 컨텍스트에 관리되는 객체 기준 건건이 삭제
	deleteAllById(Iterable<ID> ids)	void	여러 ID에 해당하는 데이터들을 삭제	루프를 돌며 건건이 deleteById를 호출
	deleteAll(Iterable<? extends T>)	void	주어진 엔티티 목록을 모두 삭제	루프를 돌며 건건이 DELETE 쿼리 발생
	deleteAll()	void	테이블의 모든 데이터를 삭제	[주의] 전체 조회를 먼저 한 뒤, 건건이 DELETE 하므로 대형 장애 유발 가능
Delete Batch	deleteAllInBatch(Iterable<T>)	void	주어진 엔티티 목록을 대량으로 한 번에 삭제합니다.	[추천] 선조회 없이 WHERE id IN (...) 단 한 번의 쿼리로 처리
	deleteAllByIdInBatch(Iterable<ID>)	void	주어진 ID 목록에 해당하는 데이터를 한 번에 삭제합니다.	[추천] 엔티티 객체 없이 ID 리스트만으로 단 하나의 DELETE 쿼리 수행
	deleteAllInBatch()	void	테이블의 모든 데이터를 통째로 삭제합니다.	[추천] 선조회 없이 DELETE FROM 테이블 단방향 벌크 쿼리 실행
Etc.	count()	long	테이블의 전체 데이터 개수를 반환합니다.	select count(*) 쿼리 발생
	flush()	void	영속성 컨텍스트의 변경 내용을 DB에 동기화합니다.	트랜잭션은 유지하되 쿼리만 미리 밀어낼 때 사용

4. Persistence : 공통 Repository(2)

InfiniTINN Platform
Development



공통 Repository 기능 설명 - AmsRepository

분류	메서드명	주요 기능 설명	특징 (Lock / Usable 조건)
단건 조회	findById	ID로 특정 엔티티를 조회	No Lock
	findByIdForUpdate	ID로 특정 엔티티를 조회	DB Lock 적용 (수정용 조회)
	getUsableById	활성화된(Usable) 엔티티 1건을 조회	No Lock + Usable 조건
	getUsableByIdForUpdate	활성화된(Usable) 엔티티 1건을 조회	DB Lock 적용 + Usable 조건
목록 조회	findList	검색 조건(Map)에 맞는 엔티티 목록을 조회	No Lock
	findListForUpdate	검색 조건(Map)에 맞는 엔티티 목록을 조회	DB Lock 적용 (수정용 조회)
	getList	검색 조건에 맞는 활성화된(Usable) 엔티티 목록을 조회	No Lock + Usable 조건
	getListForUpdate	검색 조건에 맞는 활성화된(Usable) 엔티티 목록을 조회	DB Lock 적용 + Usable 조건
	getAllUsable	활성화된 모든 엔티티 목록을 조회	Usable 조건 (전체 조회)
Insert Batch	insertBatch	여러 건의 데이터를 한 번에 일괄 삽입(Batch Insert)	성능 최적화용, 이력성 데이터 insert 만 하는 경우 사용
저장 & 이력	saveWithHist	엔티티를 저장하면서 이력(History) 저장 여부를 선택	기본값: 이력 함께 저장(true)
	saveAllWithHist	여러 엔티티를 저장하면서 이력(History)을 함께 저장	다건 처리
Soft Delete	makeUnUsableWithHist	엔티티를 사용 불가(UnUsable) 상태로 변경하고 이력을 남김	논리적 삭제(Soft Delete) 성격

4. Persistence : 공통 Repository(3)

InfiniTINN Platform
Development



공통 Repository 기능 설명 – AmsRepository

고려 사항	내용	구현 내용
Hist Table 저장	Hist Table이 항상 필요한 Table의 데이터 저장	saveWithHist method 이용하여 저장
	1) 현재 데이터를 이력에 저장	
	2) 일부 Column은 직전 데이터 저장 필요	prev_xxxx column 이 있으면 직전 데이터 값을 자동으로 넣어준다
Extended Column	표준 기능에서 설계된 table외에 custom column 이 추가되는 경우	예를 들면, pom_lot이라는 table에 area_id column이 추가된다고 가정하면
	1. Extended Entity 생성	0) 표준 Logic에는 extension entity를 감안하여 미리 준비 Class<? extends Lot> lotClass = entityFactory .getExtensionClass(Lot. class);
		1) Custom Entity 생성 : 기존 Lot inheritance LotExt
		2) Entity 내용 : @DiscriminatorValue("EXT") public class LotExt extends Lot { @Column(name = " areaid ", length = 40) private String areaid ; }
		3) Custom logic에서 처리하거나
	2. UserProperty map 활용	4) 각 Entity마다 UserProperty map이 있으므로 Client에서 메시지에 columnName과 value를 넣으면, 해당 column을 update한다.

5. Interface : Interface 방법-Message

InfiniTINN Platform
Development



Client-Server(Service) 간 또는 Service간 통신 방법은 REST, gRPC, Kafka, MQTT등 여러가지 방법으로 interface가 가능하다. Interface를 위해서는 표준화된 메시지 Format을 활용하여야 한다.

```
public class Message<T> {  
  
    @JsonProperty("Header")  
    @JsonAlias({"HEADER", "header"})  
    public MessageHeader header = new MessageHeader();  
  
    //public T body = new MessageBody();  
    @JsonProperty("Body")  
    @JsonAlias({"BODY", "body"})  
    public T body;  
  
    public MessageReturn reply = new MessageReturn();  
}
```

```
public class MessageHeader {  
  
    private String messageCategory;  
    private String messageName;  
    private String messageType; // Primary, Secondary, Reply  
    private String traceId;  
    private String system; // 이 시스템과 연결된 Main DB를 구분하기 위한  
    private String publishedTime = LocalDateTime.now().toString();  
    private String senderId;  
    private String originalSourceTopic;  
    private String targetTopic;  
    private String language;  
}  
  
public class MessageBody {  
    @NotBlank  
    private String userId;  
    private String pwd;  
  
    private String factoryId;  
    private String companyId;  
    private String event;  
    private LocalDateTime eventTime;  
    private String eventReason;  
    private String eventComment;  
  
    private Map<String, String> userProperty = new HashMap<>();  
}  
  
public class MessageReturn {  
    private String returnCode;  
    private String returnMessage;  
    private String returnDetailMessage;  
}
```

5. Interface : Interface 방법-Message

InfiniTINN Platform
Development



통신을 하기 위해서는 gRPC 는 proto file을 반드시 만들어야 하고, REST, Kafka, MQTT등은 json으로 메시지를 만들어 통신하며, 각 메시지에 대응하는 Java class가 있어야 한다. (설계서를 이용한 자동 generate - python)

Json message example

```
"header": {
  "messageCategory": "StepJobLot",
  "messageName": "TrackIn",
  "messageType": "Primary",
  "traceId": "",
  "system": "PINN",
  "publishedTime": "",
  "senderId": "cskim",
  "originalSourceTopic": "AMS.REQ.TOPIC",
  "targetTopic": "AMS.POM.UIX",
  "language": "KO"
},
"body": {
  "userId": "cskim",
  "pwd": null,
  "factoryId": "A",
  "lots": [
    {
      "lotId": "240812001",
      "userProperty": {
        "areaId": "PHOTO07"
      }
    }
  ],
  "equipmentId": "AC02"
},
"Result": {
  "returnCode": "",
  "returnMessage": "",
  "returnDetailMessage": ""
}
```

Java message class

```
public class TrackInBody extends MessageBody {
    private List<LotBody> lots = new ArrayList<>();
    private String equipmentId; // EQUIPMENTID
    private String locationId; // LOCATIONID

    @Data
    @EqualsAndHashCode(callSuper = false)
    public static class LotBody extends MessageBody {
        @NotBlank
        private String lotId;
    }
}
```

proto example: Service

```
// PomService.proto
syntax = "proto3";
package com.ams.pom.grpc;

option java_multiple_files = true;
option java_package = "com.ams.pom.grpc";

// Body 파일들은 Python이 만들어준 걸 갖다 쓰기만 함
import "trackIn.proto";

service PomService {
    // 새로운 메시지가 추가될 때만 한 줄씩 추가
    rpc TrackIn(TrackInRequest) returns (TrackInResponse);
}
```

Message (common)

```
// Common.proto - message common
syntax = "proto3";
package com.ams.pom.grpc;
option java_multiple_files = true;
option java_package = "com.ams.pom.grpc";
message MsgHeader {
    string messageCategory = 1;
    string messageName = 2;
    string messageType = 3;
    string transactionId = 4;
    string system = 5;
    string publishedTime = 6;
    string senderId = 7;
    string originalSourceTopic = 8;
    string targetTopic = 9;
    string language = 10;
}
message MsgReturn {
    // MessageReturn Fields (Result Status)
    string returnCode = 1;
    string returnMessage = 2;
    string returnDetailMessage = 3;
}
```

Generated code

- _init_.py
- common_pb2.py
- common_pb2_grpc.py
- pomService_pb2.py
- pomService_pb2_grpc.py
- trackIn_pb2.py
- trackIn_pb2_grpc.py

proto Message 예제

```
syntax = "proto3";
package com.ams.pom.grpc;
option java_multiple_files = true;
option java_package = "com.ams.pom.grpc";
import "common.proto"; // message header, return 관리용인 common.

message TrackInRequest {
    // MessageHeader Fields - class = MsgHeader
    MsgHeader header = 1;
    TrackInBody body = 2;
}
message TrackInBody {
    // MessageBody Common Fields
    string userId = 1;
    string pwd = 2;
    string factoryId = 3;
    string companyId = 4;
    string event = 5;
    string eventTime = 6;
    string eventReason = 7;
    string eventComment = 8;
    map<string, string> userProperty = 9;

    // Business Specific Items
    repeated LotBody lotList = 10;
    string equipmentId = 11; // equipmentid
    string locationId = 12; // locationid
}
message TrackInResponse {
    // MessageHeader Fields - class = MsgHeader
    MsgHeader header = 1;

    // MessageReturn Fields (Result Status), class = MsgReturn
    MsgReturn reply = 2;
}
message LotBody {
    string lotId = 1; // lotid
    map<string, string> userProperty = 2;
}
```

6. 사용자 관리 – 권한 관리

InfiniTINN Platform
Development



Authentication vs. Authorization



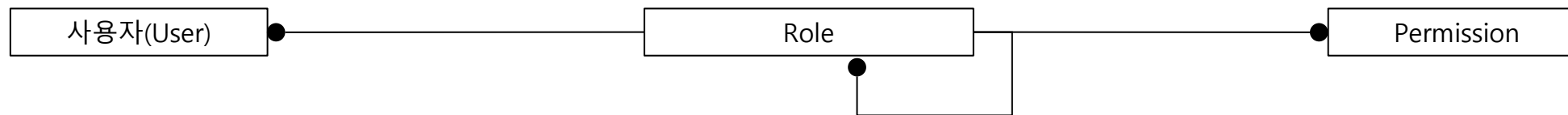
6. 사용자 관리 - 권한 관리

InfiniTINN Platform
Development

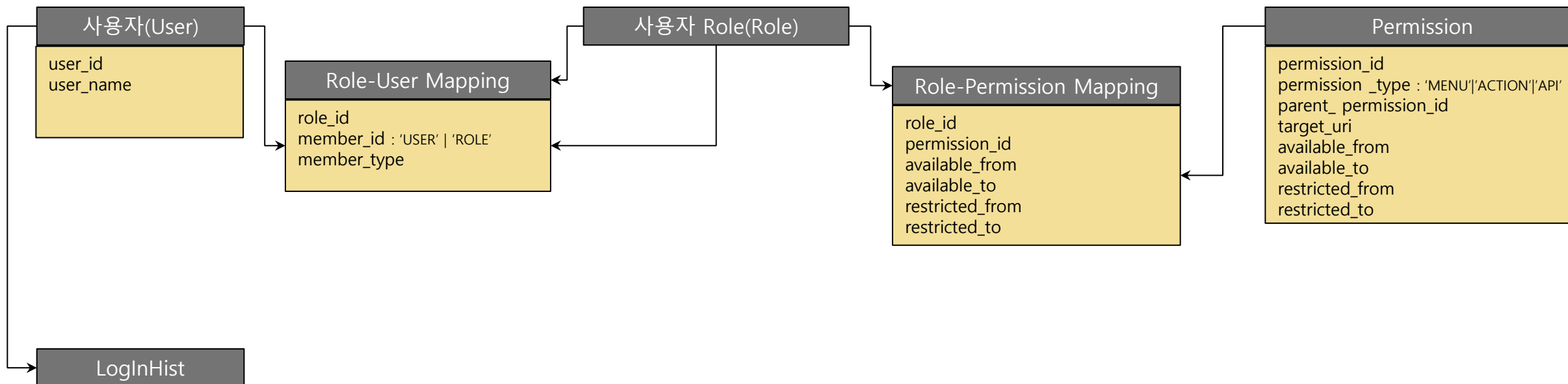


권한 관리는 RBAC(Role Based Access Control) 을 기본으로 하고, 향 후 필요시 ReBAC(Relation-Based Access Control) 추가하여 관리 필요

Object Model



Entity-Relation



6. 사용자 관리 – 암호화 방식 비교

InfiniTINN Platform
Development



암호화 – 사용자 비밀번호는 'SHA-256' 사용

비교 항목	AES-256	SHA-256
Name	양방향 암호화	단방향 암호화/해시
설명	원본이 가능한 암호화	원본이 절대 불가능한 암호화
상세 설명	암호화 키를 가지고 데이터를 잠갔다가, 나중에 동일한 키로 다시 열어 서 원본 내용을 확인할 수 있는(복호화가 가능한) 양방향 암호화 방식	한 번 암호화(해시화)하면, 지구상의 어떤 컴퓨터로도 원본 데이터를 다시 알아 낼 수 없는(복호화가 불가능한) 단방향 암호화 방식
동작방식	원본 데이터 → 암호화(Key 필요) → 암호문 → 복호화(Key 필요) → 원 본 데이터 확인 가능	원본 데이터 → 해시 변환 → 고정된 길이의 암호문 → 복호화 절대 불가
주요 사용처	데이터의 원본을 나중에 다시 읽어야 하는 항목들 (예: 고객의 계좌번호, 주민등록번호, 전화번호, 이메일 등)	사용자 비밀번호

6. 사용자 관리 – Password Policy constant

InfiniTINN Platform
Development



사용자 password 관리를 위한 설정 값 – table에 저장하여 사용

Table:
com_constant_class

class_id: 'PWD_POLICY'

Table:
com_constant_definition

class_id: 'PWD_POLICY'

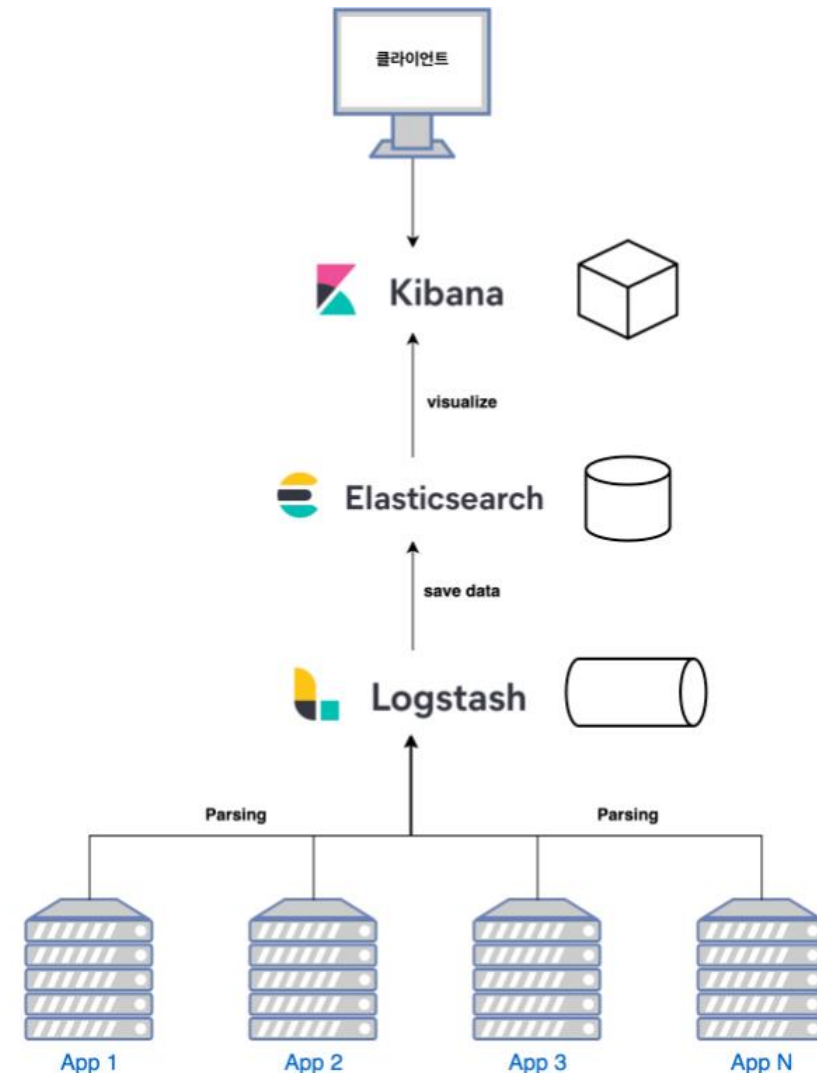
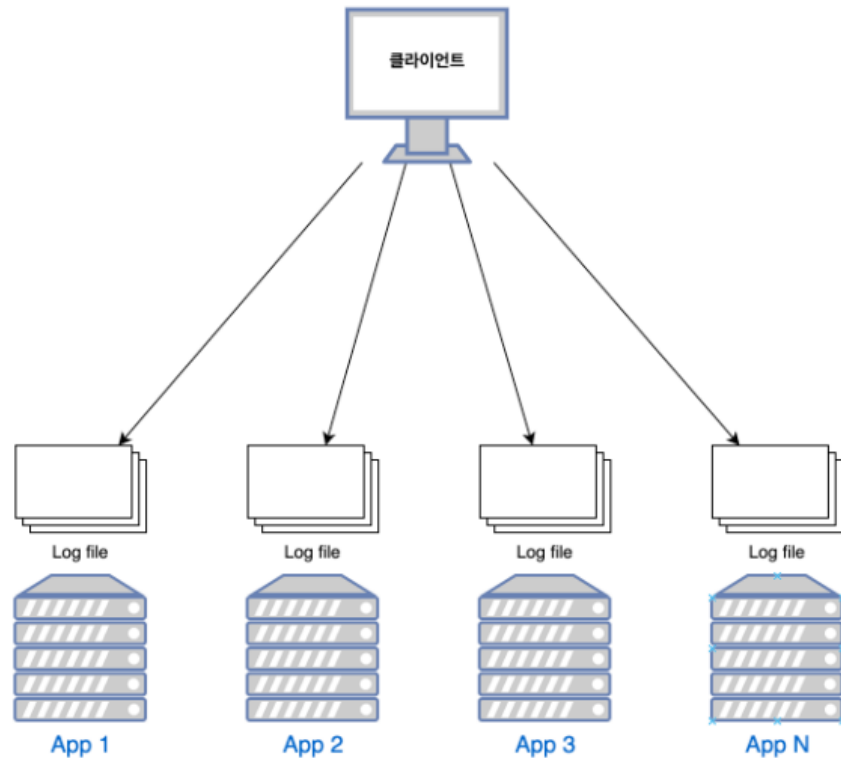
definition_code	definition_value	description
MIN_LEN	9	Password Minium length 정의
CHANGE_CYCLE	90	Password 변경 주기 정의
USE_REGEX	TRUE	Password 정규식 검증 여부
REGEX_PATTERN	^(?=.*[A-Za-z])(?=.*\d)(?=.*[~!@#\$%^&*]).+\$	Password영문/숫자/특수문자 조합 정규식 패턴 정의
REGEX_MSG	비밀번호는 영문, 숫자, 특수문자를 조합해야 합니다.	Password 에러 발생 시 사용자 안내 문구

7. 감사 추적 – 기본 log

InfiniTINN Platform
Development



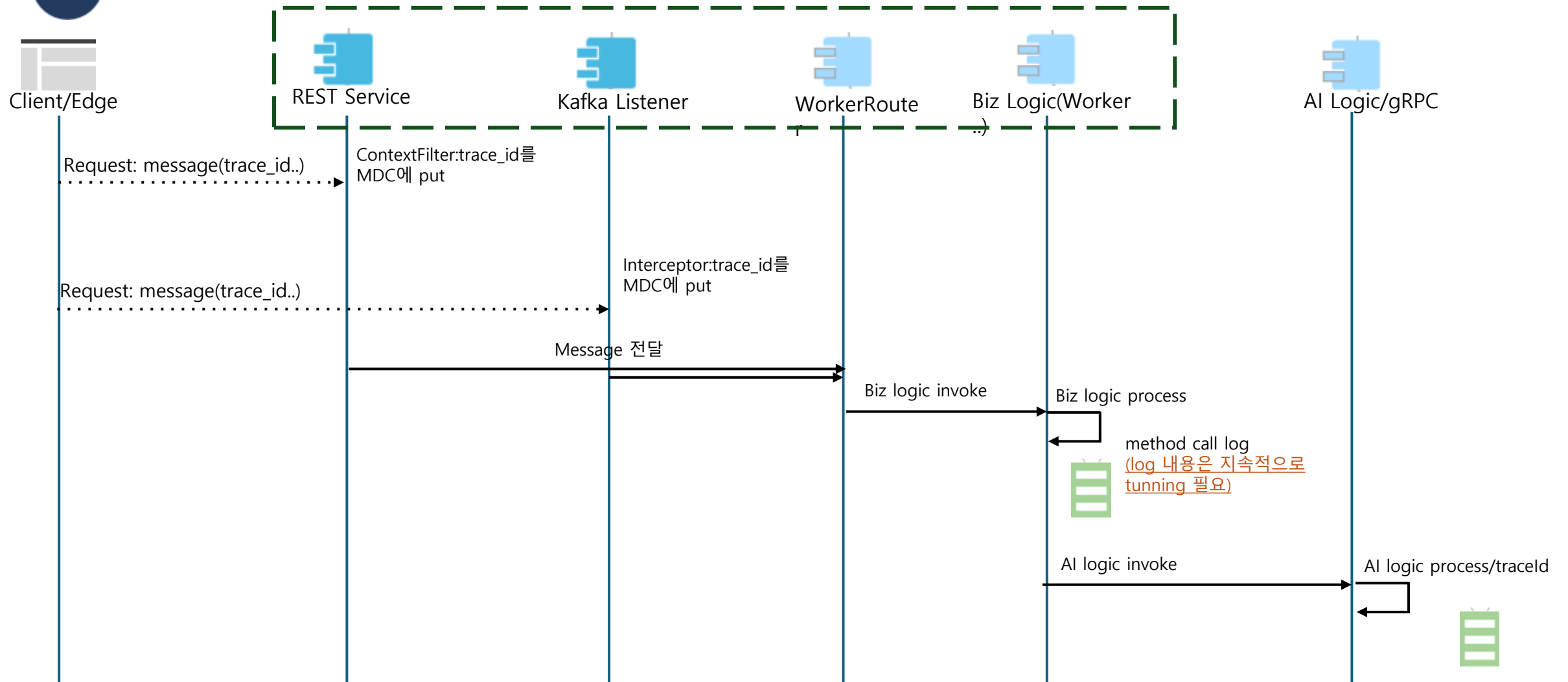
K8S로 Process를 관리하면 같은 Service라도 pod 개수만큼 log file 생김
Docker활용하는 경우에도, Container 개수만큼 log file 생김



7. 감사 추적 – 기본 log



Log는 1차로 log-back을 사용하여 File에 저장

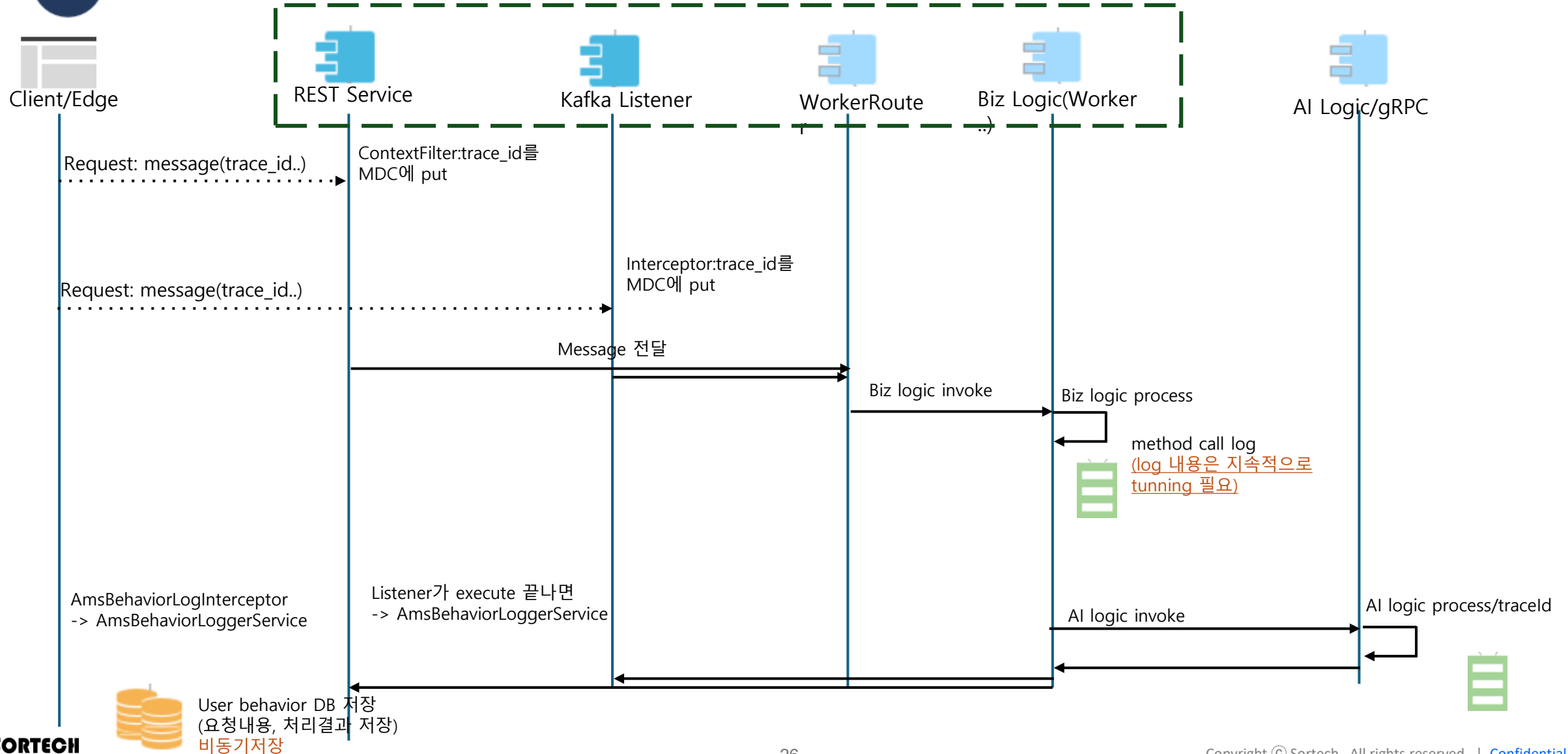


7. 감사 추적 – user behavior

InfiniTINN Platform
Development

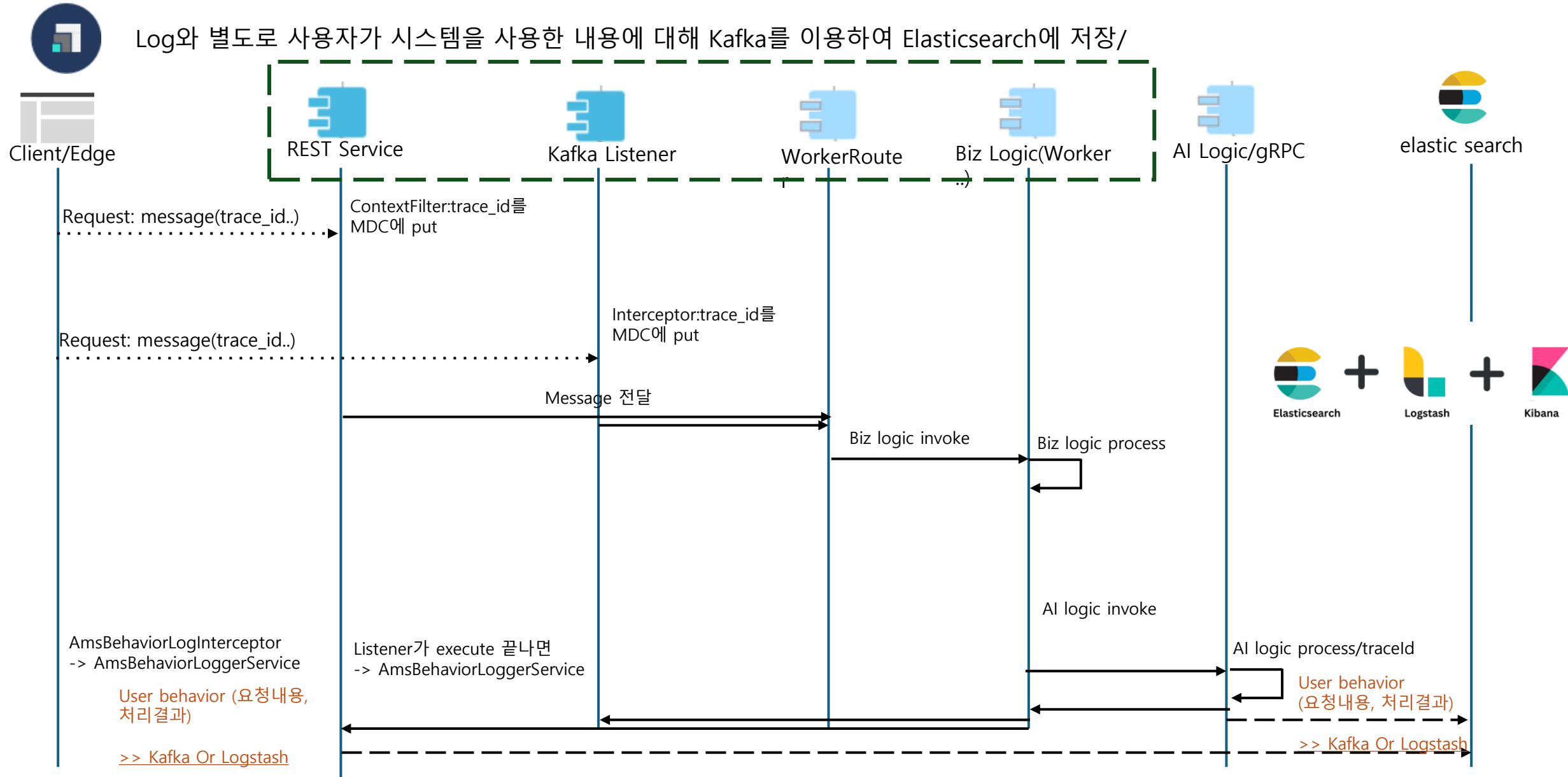


Log와 별도로 사용자가 시스템을 사용한 내용에 대해 user_behavior_hist table에 저장



7. 감사 추적 – user behavior/log – To-Be : ELK 활용

InfiniPINN Platform
Development





E.O.D